



S.G. Ganesh

The 'Struct Hack' Technique

This month we'll discuss an interesting programming technique that started as a 'hack', entered mainstream programming and then became part of the C language!

What strikes you first when you see a structure definition like this?

```
struct array {  
    int nelem;  
    int data[1];  
};
```

If you are not familiar with the technique behind the code, you might think that it is crazy. Why would anyone create an array of size 1, as in "int data[1];" here? We can as well declare it as "int data;", right?

But if you go ahead and check how the "array" struct is used, you can see some still stranger code:

```
#define ARR_SIZE 10
```

```
struct array *ptr;  
ptr->nelem = ARR_SIZE;  
ptr = malloc (sizeof (struct array) + sizeof (int) * (ARR_SIZE - 1));  
  
for (int i = 0; i < ptr->nelem; i++)  
    p->data [i] = i;
```

Can you guess what is going on now?

It is not possible to create variable size arrays in C. We can always dynamically allocate an array, but often it is convenient to have a structure where we can have one member as a variable size array.

A 'shrewd' way to overcome this limitation is to have an element in struct to remember the size of the array we want and then have an array with one element size. Then, instead of allocating memory dynamically as 'sizeof(struct array)', we calculate the size to include the amount of memory we want to allocate for the array also. But the array already had one element, so we need to adjust for that while dynamically allocating memory (it is not possible to have zero length arrays in C; it is possible in languages like Java).

We have calculated and allocated enough memory for the array and access only within the limits. The dynamically allocated block is contiguous, and compilers usually do not check for the 'out-of-bounds' access, so it

works fine in almost all C implementations!

What we just saw is called the 'struct hack' technique (also known as the 'one-element-array-trick'), which started as a 'hack' to overcome the limitation of the fixed length of structs/arrays. There are some variations; for example, instead of "int data[1];", the declaration can be "int *data;". Since this technique worked well for most implementations and it solved a practical problem in many C applications, it started to get used widely not as a hack but as a way of overcoming the C limitation. However, strictly speaking, accessing past the end of the array results in 'undefined behaviour', according to the original language definition. So, this technique was later made as part of the language.

The C99 standard added a feature called 'flexible array members', which is nothing but the 'struct hack' technique with slight differences. Instead of allocating the last member of the struct as an array of size 1, the standard allows to declare them with no mention of the size. For example, here is the same 'struct array' we saw earlier, written in C99:

```
struct array {  
    int nelem;  
    int data[]; // FLM  
};
```

The dynamic memory allocation can be done using malloc, as described earlier. But it will not consider the flexible array member while calculating the size of the struct, which makes the expression in the malloc easier.

Note that this technique is only for C. For most other languages there is no need to use this technique. For example, since C++ has better facilities, we can use vectors for variable sized (dynamically growing) arrays. 

About the author:

S G Ganesh is a research engineer in Siemens (Corporate Technology). His latest book is "60 Tips on Object Oriented Programming", published by Tata McGraw-Hill. You can reach him at sgganesh@gmail.com.